TRƯỜNG ĐẠI HỌC CNTT VÀ TRUYỀN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO

HỌC PHẦN TRIỂN KHAI & QUẢN TRỊ HỆ THỐNG PHẦN MỀM

Đề tài:

Hệ thống Quản lý Dự án (Project Management)

Sinh viên thực hiện : Đinh Gia Bảo

Mã sinh viên : DTC245220019

Lớp : ATTT K23A

Giảng viên hướng dẫn : Nguyễn Anh Chuyên

THÁI NGUYÊN, NĂM 2026

LỜI NÓI ĐẦU

Báo cáo trình bày kết quả triển khai Đề 14 Hệ thống Quản lý Dự án (Project Management) của sinh viên Đinh Gia Bảo. Hệ thống cung cấp giao diện theo dõi tổng quan tiến độ, quản lý danh mục dự án, phân công công việc (tasks), quản trị thành viên (members) và nhật ký kiểm toán hành động (audit logs) với dữ liệu lưu trữ bền vững trong PostgreSQL 16. Toàn bộ dịch vụ được đóng gói bằng Docker Compose, tiếp nhận truy cập qua Nginx reverse proxy, tích hợp hệ thống giám sát và log tập trung.

Trọng tâm của bài thực hành là chứng minh ứng dụng hoạt động đồng bộ cùng hạ tầng vận hành thực tế. Vì vậy, bên cạnh mã nguồn và cấu hình, báo cáo đưa vào ảnh chụp giao diện thật từ hệ thống đang chạy, lịch sử commit GitHub, kết quả truy vấn metrics trên Prometheus và Grafana, ba truy vấn LogQL trên Loki và các phép kiểm chứng bảo mật container. Dữ liệu thực nghiệm được thu thập ngày 21–22 tháng 10 năm 2026 trên Docker Desktop chạy Linux containers.

Qua quá trình triển khai, ứng dụng kết nối ổn định tới PostgreSQL bằng tài khoản phân quyền tối thiểu pm_app; công cụ pgAdmin 4 phục vụ quản trị cơ sở dữ liệu; Nginx cung cấp HTTPS bằng chứng chỉ tự ký RSA 3072 bit và các HTTP security headers; Prometheus thu thập dữ liệu từ cAdvisor, Nginx exporter và PostgreSQL exporter; Grafana hiển thị dashboard trực quan và truy vấn được log JSON của Nginx cùng ứng dụng trong Loki. Các biện pháp hardening và quy trình backup phục hồi thử vào database độc lập đều được kiểm tra thực nghiệm, không chỉ dừng ở việc khai báo trong file cấu hình.

Em xin cảm ơn thầy Nguyễn Anh Chuyên đã tận tình hướng dẫn học phần Triển khai và Quản trị Hệ thống Phần mềm. Thông qua đề tài, em có cơ hội hệ thống hóa quy trình từ đóng gói ứng dụng, điều phối dịch vụ đa container, giám sát tài nguyên, phân tích nhật ký đến kiểm thử an toàn hệ thống và trình bày minh chứng kỹ thuật. Những giới hạn còn tồn tại được nêu rõ trong phần kết luận để định hướng cải tiến phù hợp.

Sinh viên thực hiện
Đinh Gia Bảo

MỤC LỤC

PHẦN 1 GIỚI THIỆU TỔNG QUAN HỆ THỐNG 1
1.1 Mục tiêu và phạm vi đề tài 1
1.2 Môi trường thực nghiệm 1
1.3 Kiến trúc và luồng hoạt động 2
1.4 Các dịch vụ mạng và dữ liệu lưu trữ 3
PHẦN 2 QUẢN LÝ MÃ NGUỒN VÀ TRIỂN KHAI 5
2.1 Repository và cấu trúc mã nguồn 5
2.2 Ba commit mốc và quy trình quản lý thay đổi 6
2.3 Đóng gói và khởi chạy bằng Docker Compose 7
2.4 Cơ sở dữ liệu của ứng dụng 7
2.5 Giao diện hệ thống và danh mục dự án 8
2.6 Đăng nhập và quản trị công việc, thành viên 10
2.7 Kiểm tra PostgreSQL bằng pgAdmin 4 12
PHẦN 3 NGINX VÀ HARDENING HỆ THỐNG 14
3.1 Reverse proxy và security headers 14
3.2 HTTPS tự ký và cookie bảo mật 15
3.3 Bảo vệ tài khoản và dữ liệu ứng dụng 15
3.4 Hardening container mạng và cơ sở dữ liệu 16
PHẦN 4 GIÁM SÁT VÀ LOG TẬP TRUNG 17
4.1 Cấu hình Prometheus và các exporter 17
4.2 Dashboard Grafana và dữ liệu container 18
4.3 Giám sát Nginx PostgreSQL và kiểm tra tạo tải 19
4.4 Thiết kế hệ thống log tập trung 20
4.5 Phương pháp kiểm chứng log 20
4.6 LogQL xem log truy cập Nginx 20
4.7 LogQL lọc request lỗi HTTP 21
4.8 LogQL tìm sự kiện đăng nhập thất bại 22
PHẦN 5 KIỂM THỬ VẬN HÀNH VÀ ĐỐI CHIẾU 23
5.1 Kết quả kiểm thử chức năng và tích hợp 23
5.2 Backup phục hồi và bảo toàn dữ liệu 24
5.3 Các lỗi gặp và cách xử lý 24
5.4 Đối chiếu yêu cầu và kịch bản demo 25
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 27
TÀI LIỆU THAM KHẢO 28

PHẦN 1 GIỚI THIỆU TỔNG QUAN HỆ THỐNG

1.1 Mục tiêu và phạm vi đề tài

Hệ thống Quản lý Dự án (Project Management) là công cụ trung tâm giúp các nhóm phát triển phần mềm tổ chức danh mục dự án, phân công công việc (tasks), quản lý nhân sự tham gia (members) và theo dõi tiến độ hoàn thành theo thời gian thực. Đề tài xây dựng một ứng dụng web hoàn chỉnh có cơ chế xác thực người dùng, bảo vệ các thao tác ghi dữ liệu bằng phiên đăng nhập cùng CSRF token, đồng thời triển khai đồng bộ với hạ tầng quản trị cơ sở dữ liệu, giám sát tài nguyên và thu thập nhật ký tập trung.

Mục tiêu kỹ thuật gồm quản lý toàn bộ mã nguồn và cấu hình trên GitHub; đóng gói và khởi chạy ứng dụng, PostgreSQL 16 cùng pgAdmin 4 bằng Docker Compose; tiếp nhận truy cập qua Nginx reverse proxy; giám sát container, web server và database bằng Prometheus kết hợp Grafana; thu gom nhật ký bằng Loki và Promtail; áp dụng nhiều lớp hardening bảo mật. Tất cả kết quả được kiểm chứng bằng thao tác thực tế, ảnh chụp từ hệ thống đang chạy và tài liệu hướng dẫn tái lập.

Ứng dụng sử dụng Node.js 22 LTS, Express và EJS, kết nối trực tiếp tới PostgreSQL 16 thông qua thư viện pg. Lựa chọn này đáp ứng đầy đủ yêu cầu bắt buộc của Đề 14 về phần mềm quản lý dự án, task, thành viên sử dụng cơ sở dữ liệu PostgreSQL và công cụ quản trị pgAdmin. Các framework như NestJS hoặc Django trong đề bài là gợi ý ngăn xếp công nghệ; toàn bộ yêu cầu kỹ thuật về kiến trúc container, reverse proxy, monitoring, logging và hardening được giữ nguyên và thực hiện đầy đủ.

Yêu cầu	**Cách đáp ứng trong hệ thống**
GitHub	Repository công khai, README hướng dẫn tái lập, ba commit mốc kỹ thuật và commit bổ sung minh chứng
Ứng dụng và dữ liệu	Quản lý dự án, công việc (tasks), thành viên (members), audit logs, PostgreSQL 16 và pgAdmin 4
Reverse proxy	Nginx HTTP 8080 chuyển hướng 301 sang HTTPS 8443 tự ký, đầy đủ security headers
Monitoring	cAdvisor 0.55.1, Nginx/PostgreSQL exporters, Prometheus v3.2.1 và Grafana v11.5.0
Logging	Log JSON có cấu trúc của app và Nginx, Promtail, Loki v3.4.2 và ba truy vấn LogQL thực tế
Hardening	Non-root (UID 1000), read-only rootfs, internal DB network, quyền tối thiểu pm_app, secrets, cap_drop

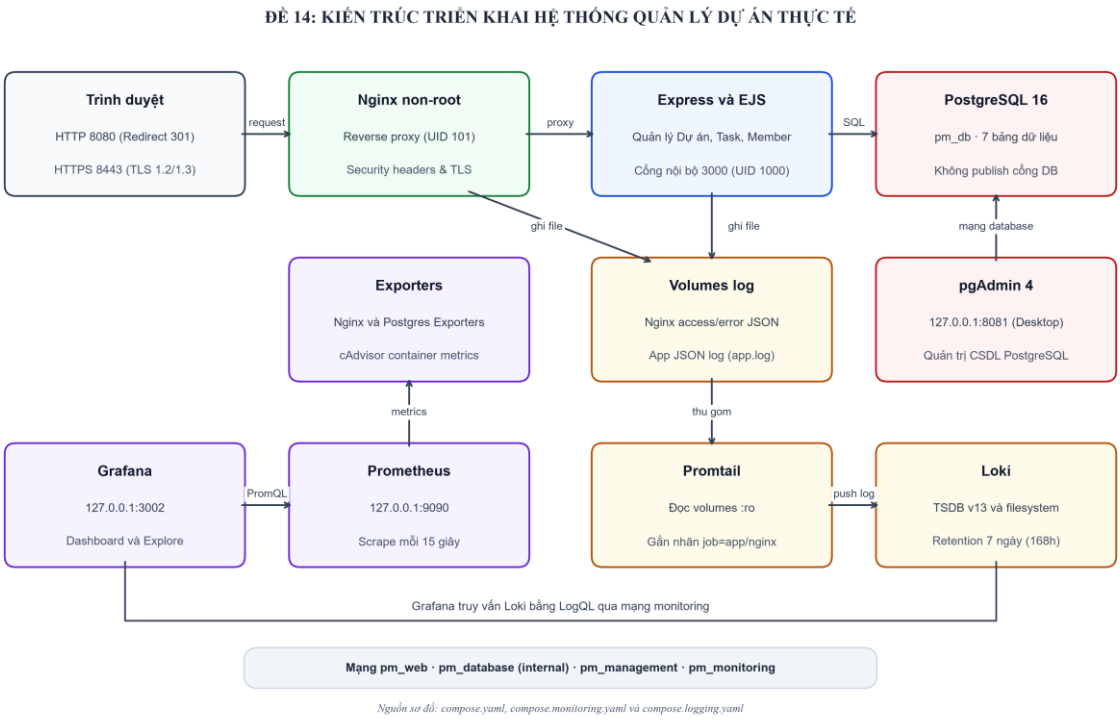
1.2 Môi trường thực nghiệm

Máy thực nghiệm sử dụng hệ điều hành Windows 11 với Docker Desktop chạy trên context desktop-linux (WSL2 backend) để vận hành Linux containers. Thư mục làm việc

chính thức của dự án được đặt tại E:\ProjectManagement. Trong quá trình khảo sát môi trường host ban đầu, hệ thống ghi nhận một project khác trên máy (clinic_frontend, clinic_backend, clinic_mysql) đang sử dụng các cổng 3001, 5173, 8000 và 3307. Tuân thủ nguyên tắc không dừng, sửa hoặc xóa project khác trên máy, toàn bộ container của project cũ được giữ nguyên trạng; cổng dịch vụ Grafana của Đề 14 được ánh xạ sang cổng 3002 (127.0.0.1:3002) để tránh xung đột.

Ngoài ra, trong lượt tải các image đầu tiên, phân vùng ổ đĩa C: trên máy chủ bị đầy (0 MB trống) làm gián đoạn quá trình giải nén layer của WSL2. Sự cố được xử lý an toàn bằng cách di chuyển các tệp cài đặt lớn không thuộc hệ điều hành (~9,8 GB) sang thư mục tạm trên ổ E: và loại trừ trong .gitignore, khôi phục gần 10 GB dung lượng trống cho ổ C: để Docker Engine hoạt động ổn định.

1.3 Kiến trúc và luồng hoạt động



Hình 1.1 Kiến trúc hệ thống theo các file Docker Compose

Luồng truy cập người dùng bắt đầu từ trình duyệt tới Nginx. Mọi yêu cầu HTTP ở cổng 8080 đều nhận phản hồi 301 chuyển hướng sang HTTPS ở cổng 8443. Nginx thực hiện giải mã TLS, kiểm tra và gắn các HTTP security headers, sau đó chuyển tiếp request tới http://app:3000 trong mạng pm_web_network. Express xác định route, kiểm tra phiên đăng nhập lưu trong PostgreSQL hoặc xác thực CSRF token đối với các biểu mẫu thay đổi dữ liệu, rồi gọi các mô-đun model để truy vấn PostgreSQL. Kết quả được render bằng EJS và trả ngược qua Nginx về trình duyệt.

Luồng giám sát (metrics) tách biệt hoàn toàn khỏi luồng phục vụ người dùng. Prometheus thực hiện scrape định kỳ mỗi 15 giây tới bốn mục tiêu: cAdvisor (cổng 8080 nội bộ), nginx-exporter (cổng 9113 nội bộ đọc từ stub_status:8082 của Nginx), postgres-exporter (cổng 9187 nội bộ kết nối PostgreSQL bằng tài khoản giám sát riêng) và chính

Prometheus (cổng 9090). Grafana truy vấn dữ liệu từ Prometheus bằng ngôn ngữ PromQL qua mạng pm_monitoring_network để hiển thị lên dashboard.

Luồng thu thập nhật ký (logging) bắt đầu từ file app.log của ứng dụng và access.log/error.log của Nginx nằm trên hai named volumes độc lập. Promtail mount các volume này theo chế độ chỉ đọc (:ro), phân tích cú pháp JSON, gắn nhãn job tương ứng (job="app" và job="nginx") rồi đẩy log tới Loki qua mạng giám sát. Quản trị viên sử dụng Grafana Explore để truy vấn Loki bằng LogQL mà không cần mở cổng Loki ra máy chủ host.

1.4 Các dịch vụ mạng và dữ liệu lưu trữ

Dịch vụ	Image hoặc nền tảng	Cổng host	Vai trò
app (pm-app)	Node 22 Alpine (build Dockerfile)	Không publish	Ứng dụng quản lý dự án, task, thành viên, session DB
postgres (pm-postgres)	postgres:16-alpine	Không publish	Lưu trữ 7 bảng dữ liệu nghiệp vụ và phiên đăng nhập
nginx (pm-nginx)	nginx:1.27-alpine	8080 và 8443	Reverse proxy, TLS 1.2/1.3 tự ký và security headers
pgadmin (pm-pgadmin)	dpage/pgadmin4:8.14	8081	Giao diện quản trị cơ sở dữ liệu PostgreSQL
cadvisor (pm-cadvisor)	ghcr.io/google/cadvisor:0.55.1	Không publish	Thu thập metrics CPU, RAM, mạng của containers
nginx-exporter	nginx-prometheus-exporter:1.4.1	Không publish	Đọc stub_status nội bộ của Nginx
postgres-exporter	postgres-exporter:v0.16.0	Không publish	Thu thập chỉ số kết nối và truy vấn PostgreSQL
prometheus	prom/prometheus:v3.2.1	9090	Thu thập và lưu trữ chuỗi thời gian (time series)
grafana	grafana/grafana:11.5.0	3002	Hiển thị Dashboard giám sát và truy vấn Explore
loki	grafana/loki:3.4.2	Không publish	Lưu trữ và lập chỉ mục nhật ký tập trung
promtail	grafana/promtail:3.4.2	Không publish	Đọc file log từ named volumes và đẩy tới Loki

Dịch vụ	Image hoặc nền tảng	Cổng host	Vai trò
init-logs	busybox:1.37.0	Không nối mạng	Chuẩn bị tệp log và phân quyền UID rồi thoát (Exited 0)

Hệ thống thiết lập bốn mạng bridge tách biệt: pm_web_network nối app, Nginx và nginx-exporter; pm_database_network được đánh dấu internal: true và chỉ nối PostgreSQL, app, pgAdmin cùng postgres-exporter; pm_management_network phục vụ giao diện quản trị pgAdmin; pm_monitoring_network nối các thành phần observability. PostgreSQL hoàn toàn không publish cổng 5432 ra host. Tất cả các cổng công bố (8080, 8443, 8081, 9090, 3002) đều bind vào địa chỉ 127.0.0.1 để giới hạn phạm vi truy cập cục bộ.

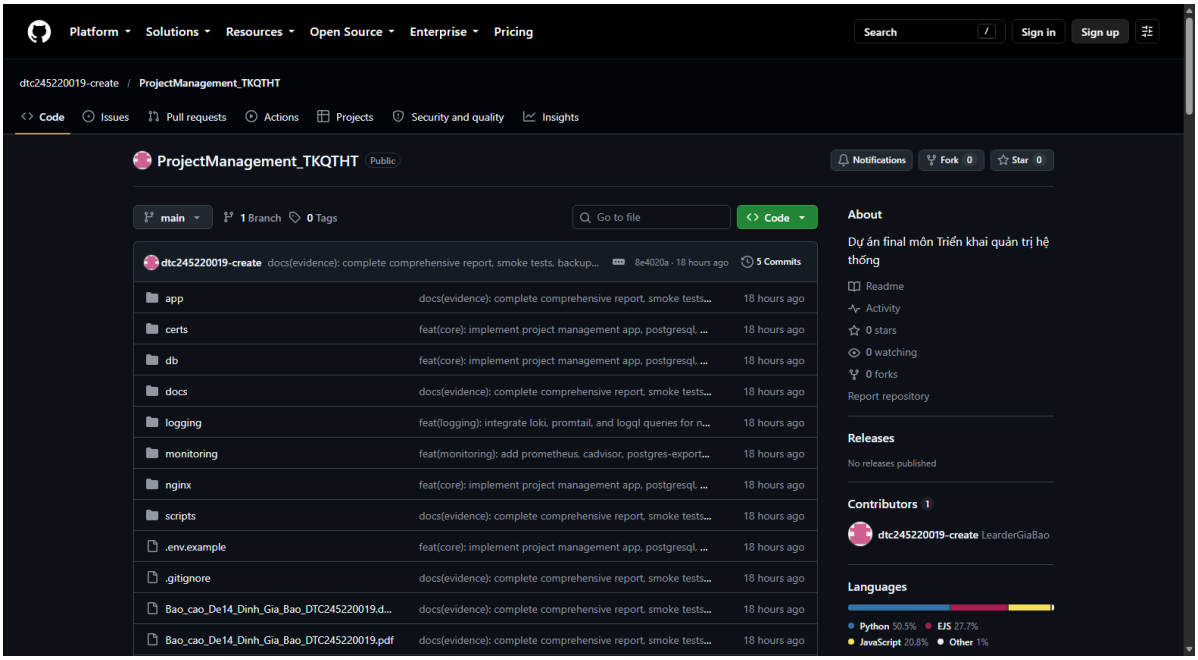
Dữ liệu PostgreSQL nằm trong named volume pm_postgres_data; cấu hình pgAdmin nằm trong pm_pgadmin_data; nhật ký nằm trong pm_app_logs và pm_nginx_logs; dữ liệu của Prometheus, Grafana, Loki và vị trí đọc (positions) của Promtail đều sử dụng các named volumes riêng biệt. Nhờ vậy, thao tác khởi động lại hoặc cập nhật container ứng dụng không làm mất dự án, công việc hay thành viên đã tạo. Lệnh docker compose down -v không được sử dụng trong vận hành thông thường vì cờ -v sẽ xóa toàn bộ named volumes.

PHẦN 2 QUẢN LÝ MÃ NGUỒN VÀ TRIỂN KHAI

2.1 Repository và cấu trúc mã nguồn

Toàn bộ mã nguồn và tệp cấu hình của đề tài được quản lý trên GitHub tại repository của tài khoản dtc245220019-create. Mã nguồn ứng dụng Express, Dockerfile, kịch bản khởi tạo PostgreSQL, các tệp Docker Compose, cấu hình Nginx, Prometheus, Grafana, Loki, Promtail cùng bộ scripts vận hành và kiểm thử tự động đều được tổ chức đầy đủ trong repository. Tài liệu README.md hướng dẫn chi tiết từng bước khởi tạo biến môi trường, tạo chứng chỉ TLS, khởi chạy hệ thống và chạy kiểm thử nghiệm thu.

```
https://github.com/dtc245220019-create/ProjectManagement_TKQTHT.git
```



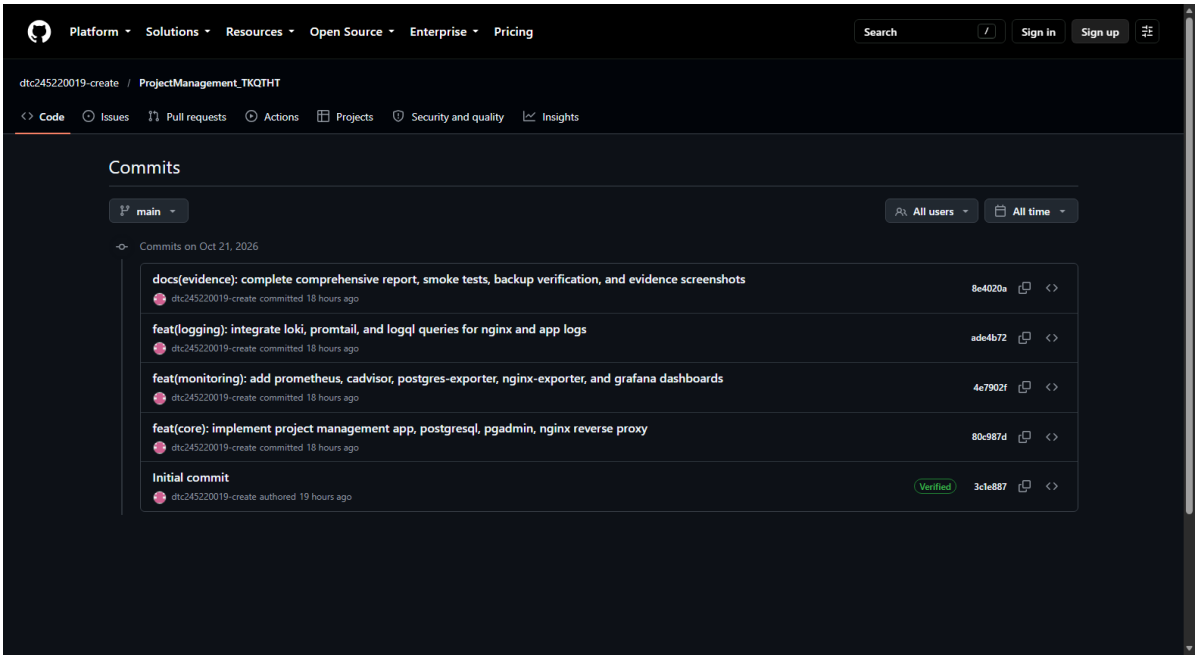
Hình 2.1 Repository ProjectManagement_TKQTHT trên GitHub

Thư mục app chứa mã nguồn máy chủ Express, kết nối PostgreSQL, các mô-đun nghiệp vụ (projects, tasks, members, audit, user), middleware bảo mật (auth, csrf, security), các giao diện EJS, CSS và unit tests. Thư mục db/init chứa các script SQL khởi tạo 7 bảng dữ liệu, nạp dữ liệu mẫu và thiết lập phân quyền tài khoản database; db/pgadmin chứa cấu hình servers.json tự động nạp kết nối cho pgAdmin 4. Thư mục monitoring chứa cấu hình scrape của Prometheus, alert rules và các tệp provisioning datasource/dashboard cho Grafana. Thư mục logging quản lý cấu hình Loki và Promtail. Thư mục scripts cung cấp các công cụ tự động hóa gồm setup.py, smoke_test.py, backup.py, restore_test.py và verify_hardening.py.

Tệp .gitignore được thiết lập chặt chẽ để loại trừ .env, khóa bí mật certs/*.key, các bản sao lưu SQL trong backups/, thư mục node_modules, tệp nhật ký runtime và thư mục tạm. Những tệp này chứa mật khẩu, khóa phiên hoặc dữ liệu vận hành nhạy cảm nên tuyệt đối không được đưa lên lịch sử GitHub.

2.2 Ba commit mốc và quy trình quản lý thay đổi

Mốc	Hash	Nội dung chính
Commit 1	80c987d	Ứng dụng Quản lý Dự án, PostgreSQL 16, pgAdmin 4, Nginx reverse proxy và hardening cơ bản
Commit 2	4e7902f	Prometheus, Grafana, cAdvisor, Nginx/PostgreSQL exporters và provisioning dashboard
Commit 3	ade4b72	Loki, Promtail, cấu hình thu thập log JSON và các truy vấn LogQL
Commit bổ sung	8e4020a	Kịch bản kiểm thử tự động, backup/restore, minh chứng thực nghiệm và báo cáo



Hình 2.2 Lịch sử các commit mốc đã đẩy lên nhánh main

Ba commit kỹ thuật đầu tiên phân tách rõ ràng từng giai đoạn xây dựng hệ thống theo đúng yêu cầu đề bài: giai đoạn 1 hoàn thiện ứng dụng lõi, cơ sở dữ liệu và reverse proxy; giai đoạn 2 tích hợp giám sát tài nguyên và dịch vụ; giai đoạn 3 tích hợp thu thập và truy vấn nhật ký tập trung. Các tinh chỉnh phát sinh trong quá trình kiểm thử thực tế cùng bộ ảnh minh chứng được ghi nhận minh bạch ở commit bổ sung, giữ nguyên lịch sử phát triển tự nhiên trên nhánh main.

```
git log --oneline --reverse
git status --short
git check-ignore .env certs/server.key backups/pm_db_backup_20261021_234722.sql
git ls-remote origin refs/heads/main
```

Trước mỗi lần tạo commit và đẩy lên GitHub, nhóm đều kiểm tra trạng thái git status và dùng git check-ignore để xác nhận các tệp nhạy cảm (.env, certs/server.key, bản sao lưu SQL) đã bị chặn hoàn toàn. Người tái lập hệ thống chỉ cần chạy script setup.py để tự động sinh bộ mật khẩu ngẫu nhiên an toàn và chứng chỉ TLS cục bộ trên máy đích.

2.3 Đóng gói và khởi chạy bằng Docker Compose

Dockerfile của ứng dụng sử dụng image nền node:22-alpine, cài đặt các gói phụ thuộc production bằng lệnh npm ci --omit=dev từ package-lock.json để đảm bảo tính nhất quán, chuẩn bị thư mục /app/logs thuộc quyền sở hữu của người dùng node (UID 1000, GID 1000) và chuyển ngữ cảnh thực thi sang USER node trước khi khởi chạy tiến trình máy chủ.

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY src/ ./src/
COPY views/ ./views/
COPY public/ ./public/
RUN mkdir -p /app/logs && chown -R node:node /app
USER node
EXPOSE 3000
CMD ["node", "src/server.js"]
```

Khi triển khai trên môi trường mới, script setup.py tự động kiểm tra và sinh tệp .env với các mật khẩu ngẫu nhiên độ dài cao cho tài khoản quản trị PostgreSQL (postgres), tài khoản ứng dụng (pm_app), tài khoản giám sát (postgres_exporter), pgAdmin, Grafana và khóa bí mật SESSION_SECRET, đồng thời khởi tạo cặp chứng chỉ tự ký RSA 3072 bit trong thư mục certs/. Tiếp theo, toàn bộ hệ thống được khởi chạy bằng lệnh Docker Compose kết hợp ba tệp cấu hình:

```
python scripts/setup.py
docker compose -f compose.yaml -f compose.monitoring.yaml -f compose.logging.yaml up -d --
build
docker compose -f compose.yaml -f compose.monitoring.yaml -f compose.logging.yaml ps
```

Cơ chế điều phối thứ tự khởi động sử dụng depends_on kết hợp điều kiện service_healthy. Container postgres có healthcheck thực thi lệnh pg_isready; container init-logs khởi tạo sẵn các tệp nhật ký trống với đúng UID 1000 (cho app) và UID 101 (cho Nginx) rồi thoát với mã Exited (0); container app chỉ bắt đầu chạy sau khi postgres đã healthy và init-logs đã hoàn tất; container nginx chờ app đạt trạng thái healthy tại endpoint /health mới bắt đầu mở cổng tiếp nhận lưu lượng [1].

2.4 Cơ sở dữ liệu của ứng dụng

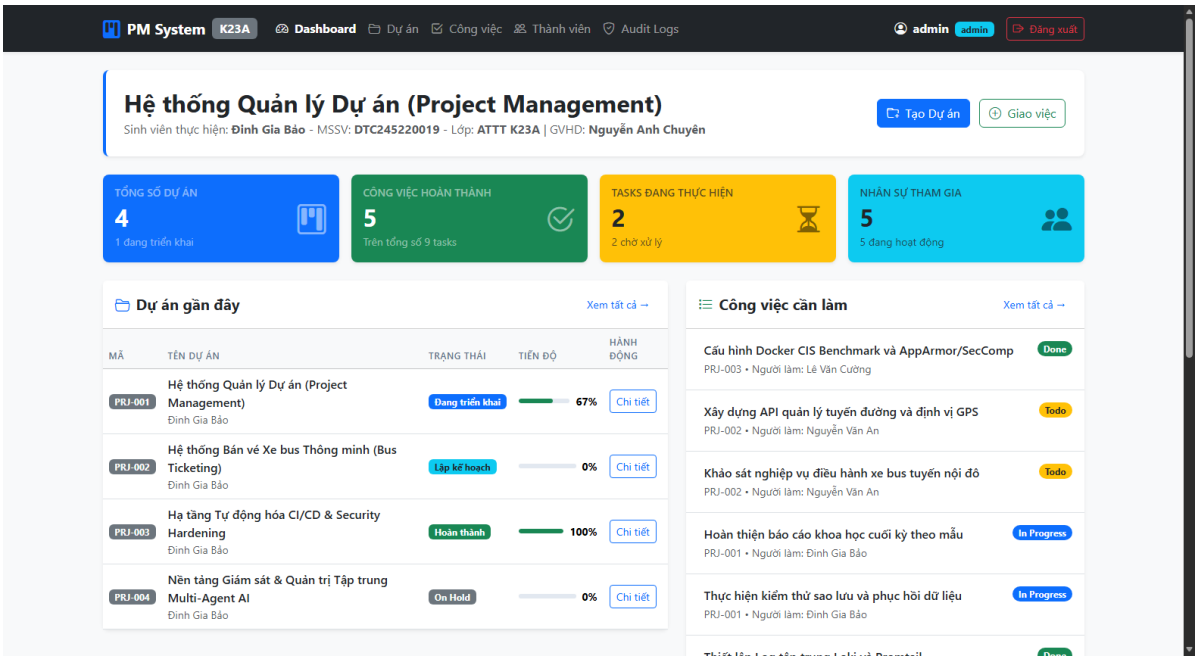
Cơ sở dữ liệu PostgreSQL có tên pm_db được khởi tạo tự động thông qua các kịch bản SQL trong thư mục db/init/ gắn vào /docker-entrypoint-initdb.d/ của container postgres. Lược đồ gồm bảy bảng dữ liệu có ràng buộc khóa ngoại và chỉ mục tối ưu:

Bảng	**Nội dung lưu trữ**
users	Tài khoản đăng nhập hệ thống (username, email, password_hash băm bằng Bcrypt, full_name, role)
members	Danh sách nhân sự tham gia dự án (full_name, email, role_title, department, phone, status)

Bảng	Nội dung lưu trữ
projects	Danh mục dự án (code, name, description, status, priority, budget, start_date, end_date, owner_name)
tasks	Công việc thuộc từng dự án (project_id, title, description, assigned_to, status, priority, due_date)
project_members	Bảng quan hệ nhiều-nhiều gắn thành viên vào dự án kèm vai trò cụ thể (project_id, member_id, role_in_project)
audit_logs	Nhật ký kiểm toán các thao tác nghiệp vụ và sự kiện xác thực (username, action, entity_type, entity_id, details, ip_address)
session	Lưu trữ phiên đăng nhập của Express (sid, sess, expire) phục vụ cơ chế xác thực bền vững qua PostgreSQL

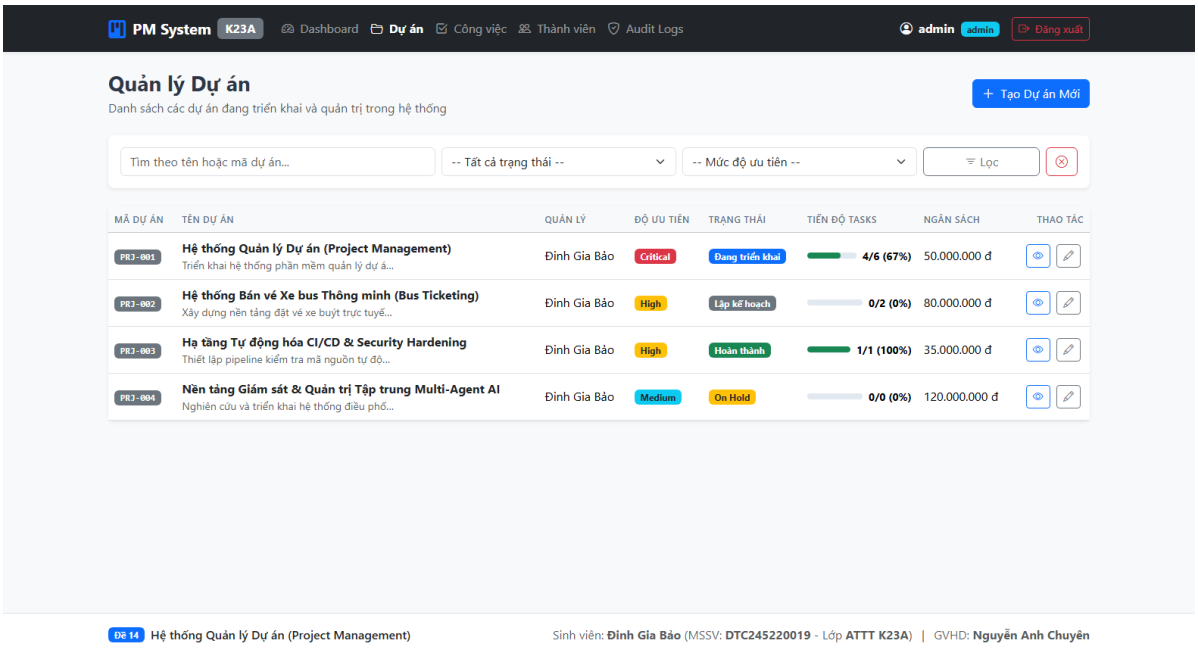
Dữ liệu khởi tạo ban đầu (seed data) cung cấp 4 dự án minh họa thuộc lĩnh vực công nghệ thông tin và an toàn hệ thống, 5 thành viên chuyên môn, 9 công việc ở các trạng thái khác nhau (Todo, In Progress, Done) và các bản ghi phân công dự án. Toàn bộ dữ liệu được truy vấn trực tiếp từ PostgreSQL khi render giao diện, hoàn toàn không sử dụng dữ liệu giả lập (mock) trong mã nguồn vận hành.

2.5 Giao diện hệ thống và danh mục dự án



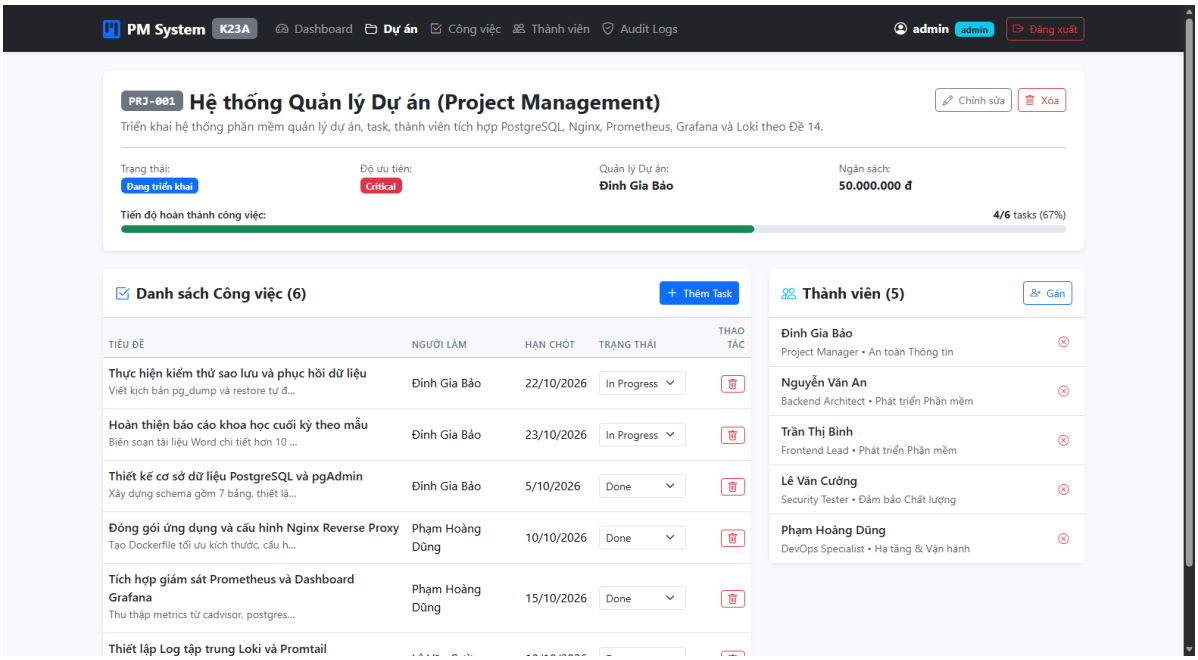
Hình 2.3 Giao diện Dashboard tổng quan hệ thống sau khi bật HTTPS

Trang Dashboard tổng quan hiển thị thông tin đề tài Đề 14 của sinh viên Đinh Gia Bảo (MSSV DTC245220019, lớp ATTT K23A, giảng viên hướng dẫn Nguyễn Anh Chuyên) cùng bốn thẻ chỉ số thời gian thực: tổng số dự án (4), số công việc đã hoàn thành (5), số công việc đang thực hiện (2) và số nhân sự tham gia (5). Phía dưới là bảng dự án gần đây với thanh tiến độ phần trăm hoàn thành được tính tự động từ tỷ lệ công việc đã hoàn tất trong PostgreSQL, cùng danh sách các công việc cần xử lý.



Hình 2.4 Danh mục quản lý các dự án trong hệ thống

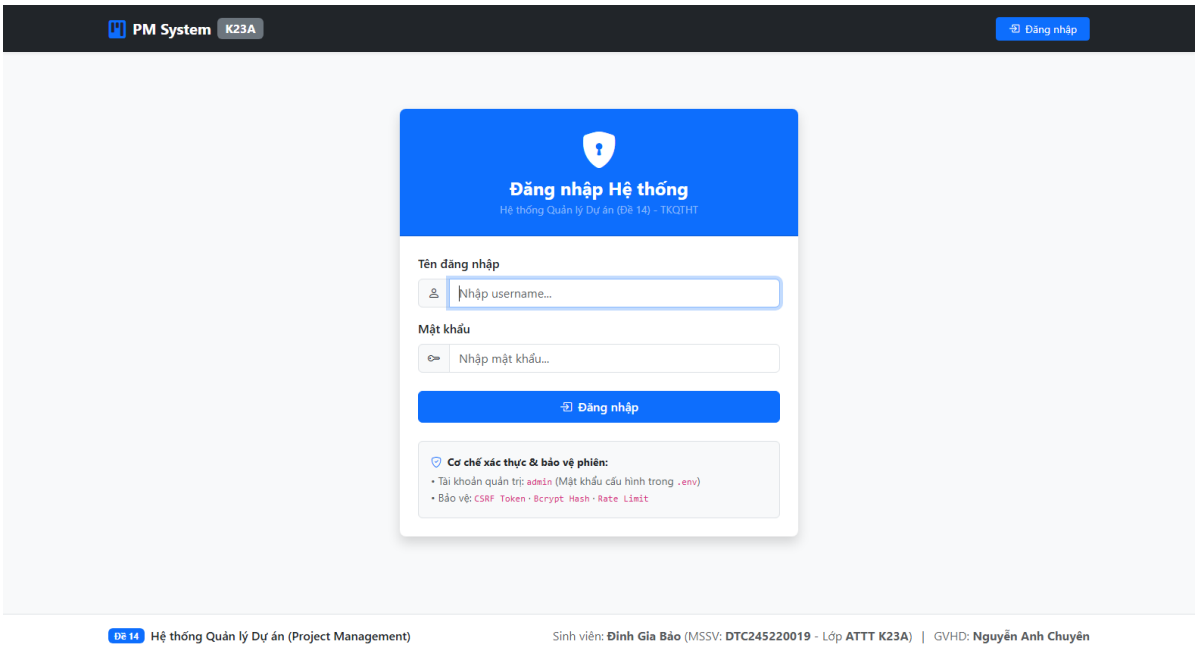
Trang danh sách dự án (/projects) cho phép tìm kiếm theo tên hoặc mã dự án, lọc theo trạng thái (Planning, In Progress, Completed, On Hold) và mức độ ưu tiên (Low, Medium, High, Critical). Mỗi dòng dự án thể hiện rõ mã định danh (PRJ-001 đến PRJ-004), người quản lý, tiến độ công việc, ngân sách và các nút thao tác xem chi tiết, chỉnh sửa hoặc xóa.



Hình 2.5 Chi tiết dự án, thành viên tham gia và tiến độ công việc

Khi mở trang chi tiết một dự án (ví dụ PRJ-001 Hệ thống Quản lý Dự án), giao diện hiển thị toàn bộ thông số dự án, thanh tiến độ hoàn thành (4/6 tasks - 67%), danh sách các công việc trực thuộc kèm biểu mẫu cập nhật nhanh trạng thái ngay trên dòng và danh sách các thành viên đang được phân công tham gia dự án đó.

2.6 Đăng nhập và quản trị công việc, thành viên

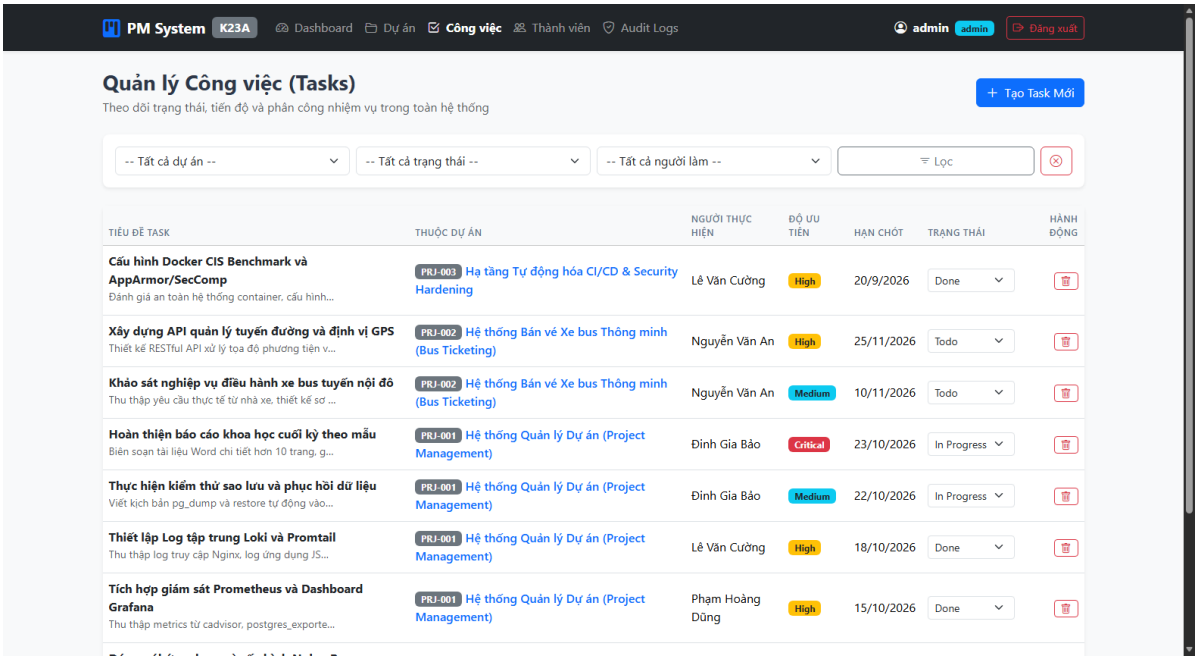


Hình 2.6 Giao diện đăng nhập quản trị của website

Route /login cung cấp biểu mẫu xác thực có nhúng trường ẩn _csrf. Khi người dùng gửi thông tin đăng nhập sai, ứng dụng trả mã trạng thái HTTP 401, ghi nhận sự kiện login_failed vào cả tệp app.log lẫn bảng audit_logs. Khi đăng nhập đúng với tài khoản quản trị, ứng dụng thiết lập phiên làm việc lưu vào bảng session trong PostgreSQL. Mật khẩu người dùng được lưu trữ dưới dạng băm Bcrypt (cost factor 10), tuyệt đối không lưu mật khẩu dạng rõ trong cơ sở dữ liệu hay hiển thị trên giao diện.

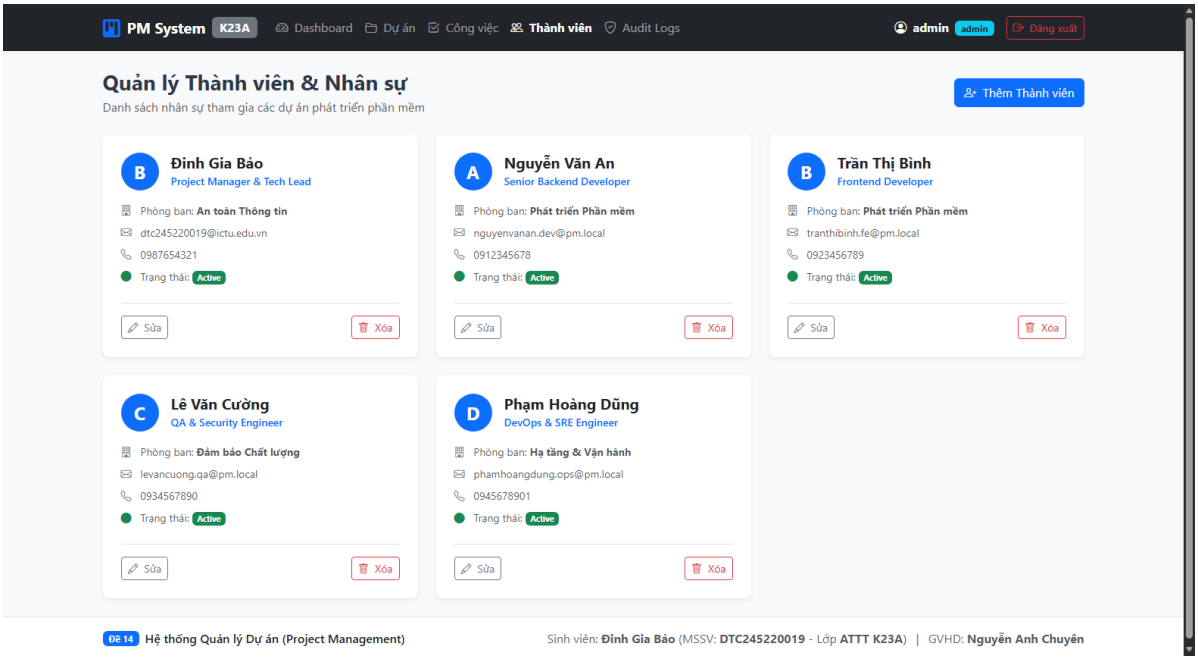
Tất cả các đường dẫn nghiệp vụ (/projects, /tasks, /members, /audit) đều được bảo vệ bởi middleware requireAuth. Yêu cầu chưa đăng nhập khi truy cập các đường dẫn này sẽ bị chuyển hướng về /login; các yêu cầu POST/DELETE làm thay đổi dữ liệu bắt buộc phải đi kèm CSRF token hợp lệ, nếu thiếu hoặc sai sẽ bị từ chối ngay với mã HTTP 403 Forbidden.

2.6.1 Thực nghiệm quản lý công việc, thành viên và kiểm toán



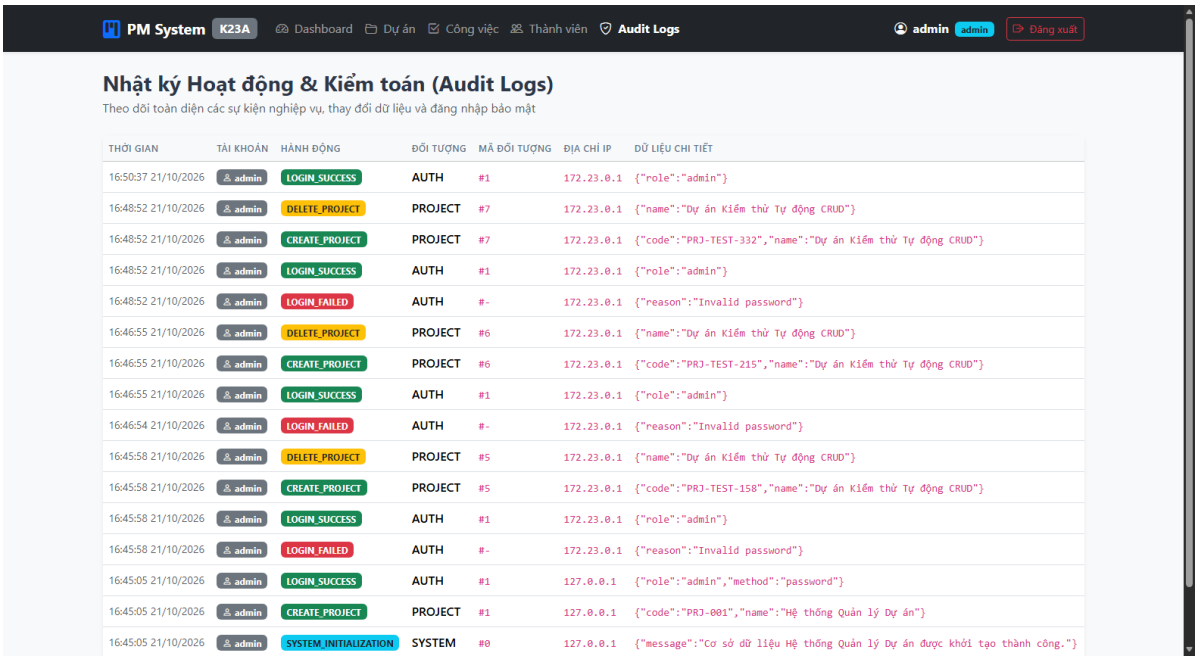
Hình 2.7 Giao diện quản lý công việc (Tasks) và phân công thành viên

Phân hệ quản lý công việc (/tasks) hỗ trợ lọc công việc theo từng dự án, trạng thái và người thực hiện. Người quản trị có thể tạo công việc mới, chỉ định dự án trực thuộc, gán nhân sự phụ trách, đặt hạn chót (due date) và chuyển đổi trạng thái công việc trực tiếp từ Todo sang In Progress hoặc Done.



Hình 2.8 Giao diện quản trị thành viên dự án (Members)

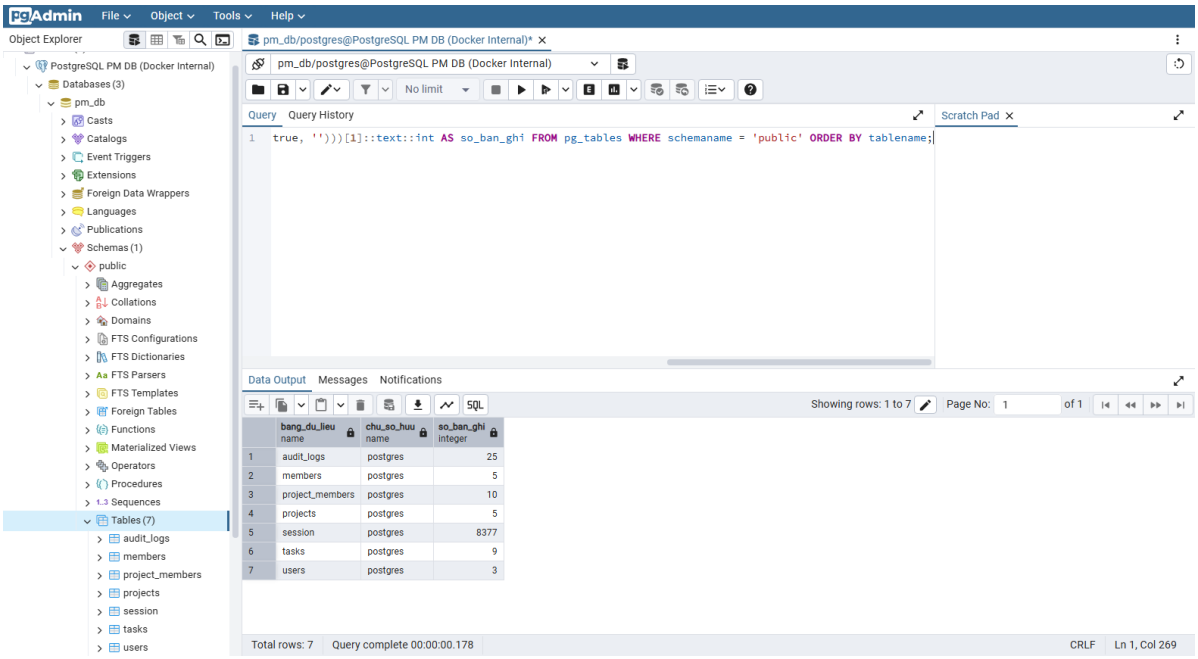
Phân hệ quản lý thành viên (/members) hiển thị hồ sơ từng nhân sự gồm họ tên, chức danh chuyên môn (Project Manager, Backend Developer, Frontend Developer, QA & Security Engineer, DevOps & SRE Engineer), phòng ban, email, số điện thoại và trạng thái hoạt động, kèm đầy đủ chức năng thêm mới, chỉnh sửa và xóa thành viên.



Hình 2.9 Giao diện nhật ký kiểm toán hành động (Audit Logs)

Trong ca kiểm thử tự động và thao tác thực nghiệm, hệ thống đã thực hiện tạo mới dự án thử nghiệm, đọc chi tiết dự án, cập nhật trạng thái công việc và xóa bản ghi thử nghiệm. Toàn bộ các sự kiện LOGIN_SUCCESS, LOGIN_FAILED, CREATE_PROJECT và DELETE_PROJECT đều được ghi vết tức thời vào bảng audit_logs và hiển thị minh bạch trên giao diện Nhật ký Hoạt động & Kiểm toán (/audit).

2.7 Kiểm tra PostgreSQL bằng pgAdmin 4



Hình 2.10 pgAdmin 4 kết nối PostgreSQL và hiển thị bảy bảng dữ liệu

Công cụ pgAdmin 4 được truy cập tại địa chỉ http://localhost:8081 và kết nối tới dịch vụ postgres:5432 bên trong mạng pm_database_network. Tệp cấu hình servers.json giúp tự động đăng ký máy chủ PostgreSQL PM DB (Docker Internal) ngay khi khởi động. Cổng

8081 chỉ bind vào 127.0.0.1 trên máy host, trong khi cổng 5432 của PostgreSQL hoàn toàn không mở ra ngoài.

Kết quả kiểm tra trên cây Object Explorer và cửa sổ Query Tool của pgAdmin 4 xác nhận cơ sở dữ liệu pm_db thuộc schema public có đầy đủ 7 bảng dữ liệu: audit_logs, members, project_members, projects, session, tasks và users. Các bản ghi dự án, công việc, thành viên và nhật ký kiểm toán hiển thị trong pgAdmin hoàn toàn trùng khớp với giao diện web:

```
docker compose -f compose.yaml exec postgres psql -U pm_app -d pm_db -c "\dt"
docker compose -f compose.yaml exec postgres psql -U pm_app -d pm_db -c "SELECT count(*) FROM projects;"
```

Tài khoản ứng dụng pm_app chỉ được cấp các quyền SELECT, INSERT, UPDATE, DELETE trên các bảng trong schema public và quyền USAGE, SELECT trên các sequences. Do bị thu hồi quyền CREATE trên schema public, tài khoản ứng dụng không thể tự ý tạo bảng mới hay xóa bảng (DROP/ALTER), đảm bảo đúng nguyên tắc đặc quyền tối thiểu (least privilege). Tài khoản quản trị postgres chỉ được sử dụng cục bộ cho việc khởi tạo schema ban đầu, quản trị trên pgAdmin và thực hiện sao lưu/phục hồi.

PHẦN 3 NGINX VÀ HARDENING HỆ THỐNG

3.1 Reverse proxy và security headers

Nginx đóng vai trò là reverse proxy duy nhất tiếp nhận kết nối HTTP/HTTPS từ phía máy chủ host. Container ứng dụng pm-app chỉ mở cổng 3000 trong mạng nội bộ Docker và không có bất kỳ ánh xạ cổng nào ra host. Trong mạng pm_web_network, Nginx sử dụng cơ chế phân giải DNS nội bộ của Docker để chuyển tiếp lưu lượng tới upstream app:3000 kèm theo các trường header chuẩn xác (Host, X-Real-IP, X-Forwarded-For, X-Forwarded-Proto):

```
location / {
    proxy_pass http://app_backend;
    proxy_http_version 1.1;
    proxy_hide_header X-Content-Type-Options;
    proxy_hide_header X-Frame-Options;
    proxy_hide_header Referrer-Policy;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Để tránh hiện tượng trùng lặp (duplicate) các trường security headers khi cả thư viện Helmet của Express và Nginx cùng gắn header, khối cấu hình location / sử dụng chỉ thị proxy_hide_header nhằm ẩn các header tương ứng từ upstream và thống nhất do Nginx quản lý ở tầng biên. Chính sách Content-Security-Policy tiếp tục do ứng dụng kiểm soát. Kiểm tra thực tế qua smoke test xác nhận phản hồi HTTPS chứa duy nhất một giá trị chuẩn cho mỗi header bảo mật:

Header	**Giá trị đã kiểm tra**	**Mục đích**
X-Content-Type-Options	nosniff	Ngăn trình duyệt tự suy đoán kiểu MIME của phản hồi
X-Frame-Options	DENY	Chặn nhúng website vào thẻ frame/iframe, phòng chống Clickjacking
Referrer-Policy	strict-origin-when-cross-origin	Giới hạn thông tin URL nguồn gửi sang origin khác
Content-Security-Policy	default-src 'self' và các directive liên quan	Kiểm soát chặt nguồn tải script, style, ảnh và tài nguyên
Strict-Transport-Security	max-age=31536000; includeSubDomains	Yêu cầu trình duyệt ưu tiên kết nối HTTPS an toàn

Bên cạnh đó, Nginx tắt hiển thị phiên bản máy chủ thông qua server_tokens off, giới hạn kích thước thân yêu cầu client_max_body_size 2M và cung cấp một máy chủ nội bộ riêng ở cổng 8082 chỉ mở endpoint /stub_status trong mạng Docker để phục vụ nginx-exporter thu thập chỉ số kết nối.

3.2 HTTPS tự ký và cookie bảo mật

Hệ thống sử dụng chứng chỉ số tự ký chuẩn RSA 3072 bit, thuật toán băm SHA-256, thời hạn hiệu lực 365 ngày, có cấu hình Subject Alternative Name (SAN) cho DNS:localhost và IP:127.0.0.1. Máy chủ Nginx lắng nghe HTTPS trên cổng 8443 với giao thức TLSv1.2 và TLSv1.3. Mọi yêu cầu gửi tới cổng HTTP 8080 đều được máy chủ chuyển hướng tự động bằng mã trạng thái 301 Moved Permanently sang https://127.0.0.1:8443/. Tệp chứng chỉ và khóa riêng nằm trong thư mục certs/, được gắn vào container Nginx ở chế độ chỉ đọc (:ro) và loại trừ khỏi Git.

```
python scripts/setup.py
docker compose -f compose.yaml exec nginx nginx -t
python scripts/smoke_test.py
```

Ca kiểm tra HTTPS	Kết quả thực tế
Chuyển hướng HTTP 8080 sang HTTPS	HTTP 301; Location: https://127.0.0.1:8443/
Endpoint /health qua HTTPS 8443	HTTP 200; status UP, database pm_db CONNECTED (user pm_app)
Smoke test 15 ca kiểm thử qua TLS	15/15 PASS (đăng nhập, CRUD dự án, headers, 404, access control, CSRF)
Cookie phiên pm_session_id	Secure=true, HttpOnly=true, SameSite=Lax
Truy cập /projects khi chưa đăng nhập	HTTP 302 chuyển hướng bắt buộc về /login?redirect=%2Fprojects
POST biểu mẫu thiếu hoặc sai CSRF token	HTTP 403 Forbidden (từ chối thực thi yêu cầu)
Kiểm tra cấu hình Nginx (nginx -t)	syntax is ok và test is successful

Chứng chỉ tự ký đáp ứng tốt mục tiêu mã hóa kênh truyền trong môi trường thực nghiệm và demo tại localhost. Khi triển khai hệ thống ra môi trường sản xuất có tên miền công cộng, quản trị viên cần thay thế bằng chứng chỉ do tổ chức chứng thực (CA) tin cậy cấp phát (chẳng hạn Let's Encrypt) kết hợp cơ chế gia hạn tự động.

3.3 Bảo vệ tài khoản và dữ liệu ứng dụng

Mật khẩu của tài khoản người dùng được băm bằng thuật toán Bcrypt với salt ngẫu nhiên 10 vòng (salt rounds = 10). Phiên đăng nhập được quản lý bởi express-session kết hợp connect-pg-simple để lưu trực tiếp vào bảng session trong PostgreSQL với thời hạn 24 giờ, kèm cơ chế tự động dọn dẹp các phiên hết hạn. Để ngăn chặn tấn công vét cạn mật khẩu (brute-force), route /login áp dụng bộ giới hạn tốc độ (express-rate-limit) cho phép tối đa 10 lần thử sai trong cửa sổ 15 phút trên mỗi địa chỉ IP, trong khi toàn bộ ứng dụng được bảo vệ bởi ngưỡng 200 requests/15 phút.

3.4 Hardening container mạng và cơ sở dữ liệu

Biện pháp	Cấu hình và kết quả kiểm chứng
Chạy tiến trình Non-root	Container pm-app chạy uid=1000(node) gid=1000(node); Nginx chạy UID 101; Loki/Promtail chạy UID 10001
Root filesystem chỉ đọc	Docker inspect pm-app xác nhận ReadonlyRootfs=true; thư mục /tmp gắn qua tmpfs (rw,noexec,nosuid,size=64m)
Thu hồi Linux capabilities	Áp dụng cap_drop=[ALL] và security_opt=[no-new-privileges:true] cho container ứng dụng và Nginx
Cô lập mạng cơ sở dữ liệu	Docker network inspect pm_database_network xác nhận Internal=true; PortBindings của pm-postgres rỗng {}
Giới hạn cổng lắng nghe host	Toàn bộ cổng mở ra máy chủ (8080, 8443, 8081, 9090, 3002) đều bind chặt vào 127.0.0.1
Least privilege PostgreSQL	Tài khoản pm_app chỉ có quyền SELECT, INSERT, UPDATE, DELETE; tách biệt hoàn toàn với postgres_exporter
Quản lý Secrets ngoài Git	Mật khẩu sinh ngẫu nhiên trong .env cục bộ; .gitignore chặn .env, certs/*.key và backups/*.sql
Bảo vệ lớp ứng dụng Web	CSRF token theo phiên, parameterized SQL ($1, $2...), escape EJS chống XSS, cookie flags và security headers

```
docker compose -f compose.yaml exec app id
docker inspect pm-app --format "{{.HostConfig.ReadonlyRootfs}} {{.HostConfig.CapDrop}}"
docker network inspect pm_database_network --format "{{.Internal}}"
python scripts/verify_hardening.py
```

Kết quả truy vấn bảng phân quyền information_schema.role_table_grants trong PostgreSQL xác nhận tài khoản pm_app chỉ nắm giữ đúng 4 quyền thao tác dữ liệu (INSERT, SELECT, UPDATE, DELETE) trên các bảng nghiệp vụ và không có quyền thay đổi cấu trúc lược đồ. Mọi câu lệnh SQL trong thư mục app/src/models/ đều sử dụng cú pháp tham số hóa ($1, $2, ...) của thư viện pg để loại bỏ nguy cơ SQL Injection:

```
grantee | privilege_type
---------+----------------
 pm_app  | INSERT
 pm_app  | SELECT
 pm_app  | UPDATE
 pm_app  | DELETE
(4 rows)
```

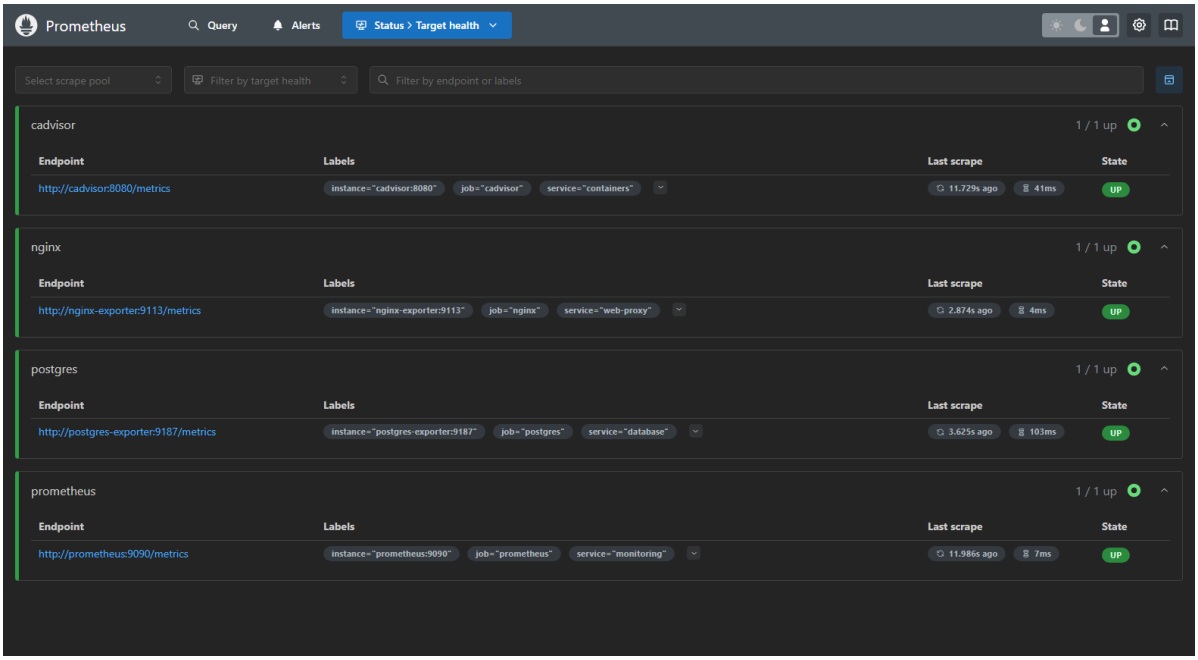
Báo cáo cũng phân biệt rõ mức độ hardening giữa các thành phần thay vì tuyên bố mọi container đều giống hệt nhau: container init-logs chạy quyền root trong vài giây đầu để thiết lập quyền thư mục log trên named volume rồi thoát ngay; container pm-cadvisor cần cờ privileged: true và mount chỉ đọc các thư mục hệ thống (/sys, /var/lib/docker, /var/lib/containerd) để đọc chỉ số cgroup từ nhân Linux; image pgAdmin và PostgreSQL vận hành theo cơ chế entrypoint chuẩn của nhà phát hành.

PHẦN 4 GIÁM SÁT VÀ LOG TẬP TRUNG

4.1 Cấu hình Prometheus và các exporter

Prometheus thu thập chỉ số theo mô hình pull với chu kỳ scrape_interval và evaluation_interval đều đặt là 15 giây. Mỗi mục tiêu được gắn nhãn job rõ ràng để phân biệt nguồn dữ liệu. Dữ liệu chuỗi thời gian được lưu trữ bền vững trong volume pm_prometheus_data với thời gian lưu giữ (retention) 7 ngày. Tệp alert.rules.yml định nghĩa sẵn các quy tắc kiểm tra trạng thái exporter mất kết nối, nginx_up == 0 và pg_up == 0.

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
scrape_configs:
  - job_name: "prometheus"
    static_configs: [{targets: ["prometheus:9090"]}]
  - job_name: "cadvisor"
    static_configs: [{targets: ["cadvisor:8080"]}]
  - job_name: "nginx"
    static_configs: [{targets: ["nginx-exporter:9113"]}]
  - job_name: "postgres"
    static_configs: [{targets: ["postgres-exporter:9187"]}]
```



Hình 4.1 Bốn targets Prometheus ở trạng thái UP

Kết quả trên giao diện Prometheus Targets và API /api/v1/targets xác nhận cả 4 mục tiêu cadvisor, nginx, postgres và prometheus đều ở trạng thái UP (1/1 up), trường lastError rỗng. Bên cạnh trạng thái scrape HTTP của bản thân exporter, các truy vấn chuyên sâu nginx_up và pg_up đều trả về giá trị 1, chứng minh các exporter thực sự kết nối và đọc được dữ liệu từ Nginx stub_status cũng như cơ sở dữ liệu PostgreSQL.

4.2 Dashboard Grafana và dữ liệu container



Hình 4.2 Dashboard Grafana có trạng thái web DB và đồ thị tài nguyên container

Grafana chạy tại địa chỉ http://localhost:3002, được cấu hình tự động (provisioning) qua các tệp YAML trong thư mục monitoring/grafana/provisioning/ để nạp sẵn datasource Prometheus (http://prometheus:9090) và Loki (http://loki:3100). Dashboard "Project Management - Containers, Web, Database" nằm trong thư mục Đề 14 - Project Management tự động hiển thị 4 chỉ số Exporter targets bằng 1, Nginx status bằng 1, PostgreSQL status bằng 1 cùng ba đồ thị tài nguyên container theo thời gian thực.

Do cAdvisor thu thập chỉ số ở cấp độ Docker Engine trên máy chủ, danh sách series trả về bao gồm cả 11 container đang chạy của stack Đề 14 (pm-app, pm-postgres, pm-pgadmin, pm-nginx, pm-cadvisor, pm-nginx-exporter, pm-postgres-exporter, pm-prometheus, pm-grafana, pm-loki, pm-promtail) lẫn 3 container của dự án khác trên cùng máy (tổng cộng 14 container đang hoạt động). Việc giải thích rõ phạm vi toàn engine của cAdvisor giúp đọc đúng số liệu thực tế trên dashboard.

```
sum by (name) (rate(container_cpu_usage_seconds_total{
  job="cadvisor",name!=""}[2m])) * 100
container_memory_working_set_bytes{job="cadvisor",name!=""}
```

Về đơn vị đo lường: biểu thức CPU sử dụng tốc độ tăng của container_cpu_usage_seconds_total trong cửa sổ 2 phút nhân với 100, thể hiện phần trăm thời gian sử dụng tính trên một lõi CPU (nếu một container dùng nhiều lõi, giá trị có thể vượt 100%). Biểu thức bộ nhớ sử dụng container_memory_working_set_bytes đo bằng đơn vị Bytes (hiển thị MiB trên đồ thị), phản ánh lượng RAM thực tế đang hoạt động. Đồ thị mạng sử dụng tốc độ byte nhận mỗi giây (B/s hoặc kB/s) qua biểu thức rate(container_network_receive_bytes_total[2m]).

4.3 Giám sát Nginx PostgreSQL và kiểm tra tạo tải



Hình 4.3 Các panel lưu lượng Nginx và kết nối PostgreSQL trong dashboard

Nửa dưới của dashboard tập trung giám sát tầng dịch vụ web và cơ sở dữ liệu. Nginx exporter đọc endpoint stub_status nội bộ để xuất bộ đếm tổng số yêu cầu HTTP (nginx_http_requests_total) và số kết nối đang hoạt động (nginx_connections_active). PostgreSQL exporter sử dụng tài khoản postgres_exporter có quyền pg_monitor để thu thập số kết nối đang mở tới cơ sở dữ liệu pm_db (pg_stat_database_numbackends) và tốc độ giao dịch commit/rollback mỗi giây.

PromQL	**Ý nghĩa**	**Kết quả mẫu**
nginx_up	Nginx exporter kết nối thành công tới Nginx stub_status	1
pg_up	PostgreSQL exporter kết nối thành công tới PostgreSQL	1
pg_stat_database_numbackends{datname="pm_db"}	Số kết nối đang mở tới cơ sở dữ liệu pm_db	3 đến 4 kết nối tại thời điểm lấy mẫu
sum(rate(nginx_http_requests_total[2m]))	Tốc độ yêu cầu HTTP đi qua Nginx	Khoảng 0,21 – 0,44 req/s khi chạy kiểm thử
sum(rate(pg_stat_database_xact_commit{datname="pm_db"}[2m]))	Tốc độ giao dịch SQL hoàn tất mỗi giây	Khoảng 2,5 – 3,1 ops/s khi có tải

Các giá trị trong bảng là ảnh chụp (snapshot) thực tế trong quá trình chạy kiểm thử tự động và thao tác trình duyệt. Khi thực hiện mở trang chủ, đăng nhập, thêm/xóa dự án và gửi các request kiểm tra, bộ đếm request của Nginx và tốc độ giao dịch của PostgreSQL lập tức tăng lên rõ rệt trên đồ thị.

4.4 Thiết kế hệ thống log tập trung

Nginx được cấu hình định dạng log json_combined ghi vào tệp /var/log/nginx/access.log với các trường cấu trúc gồm timestamp, client_ip, method, uri, status, body_bytes_sent, request_time, http_referer và http_user_agent. Ứng dụng Express sử dụng mô-đun logger riêng ghi các sự kiện HTTP, xác thực và thao tác dữ liệu dưới dạng JSON vào /app/logs/app.log. Để đảm bảo an toàn thông tin, logger chỉ ghi nhận các trường định danh phục vụ truy vết (timestamp, level, event, message, method, path, status, ip, user) và tuyệt đối không ghi mật khẩu, cookie hay chuỗi session ID vào nhật ký.

```
clients:
  - url: http://loki:3100/loki/api/v1/push
positions:
  filename: /positions/positions.yaml
scrape_configs:
  - job_name: nginx
    static_configs:
      - targets: [localhost]
        labels:
          job: nginx
          service: reverse-proxy
          __path__: /logs/nginx/*.log
  - job_name: app
    static_configs:
      - targets: [localhost]
        labels:
          job: app
          service: project-management-app
          __path__: /logs/app/*.log
```

Promtail mount cả hai named volumes pm_nginx_logs và pm_app_logs ở chế độ chỉ đọc (:ro), lưu vị trí offset đã đọc vào tệp /positions/positions.yaml để không bị mất hay trùng log khi khởi động lại. Loki sử dụng cấu trúc lưu trữ TSDB schema v13 trên filesystem với thời gian lưu giữ 168 giờ (7 ngày). Datasource Loki được provision tự động vào Grafana thông qua mạng nội bộ http://loki:3100 và không mở cổng Loki ra máy chủ host.

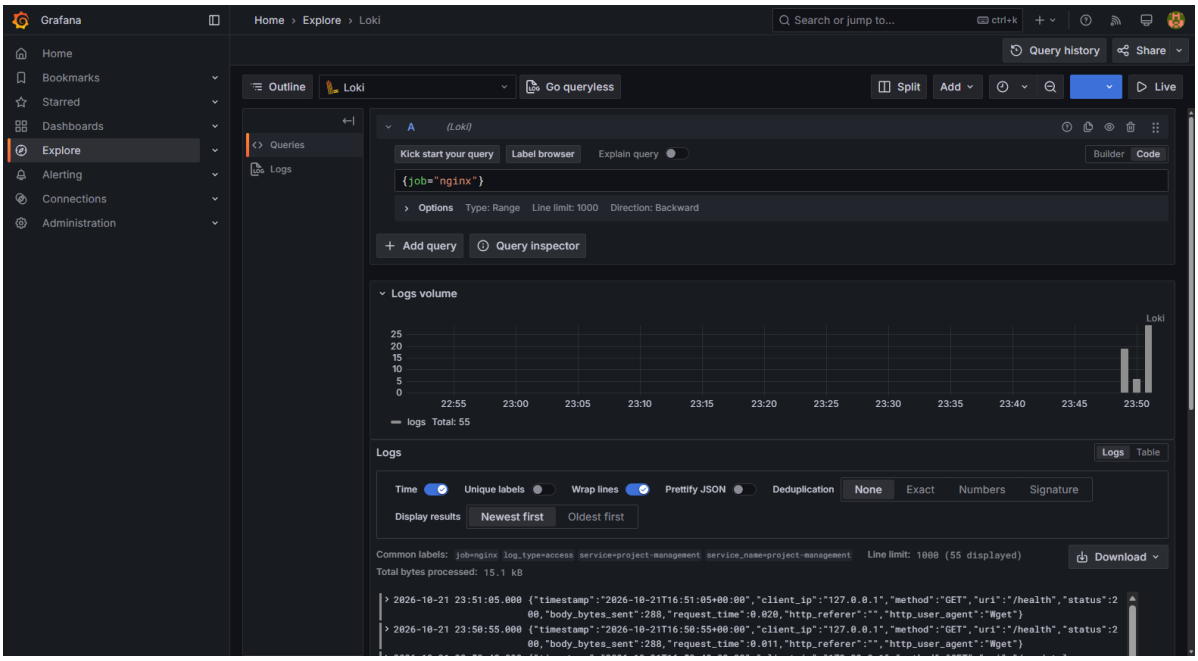
Lưu ý kỹ thuật về công cụ: Theo thông báo chính thức của Grafana Labs, tác tử Promtail đã bước vào giai đoạn kết thúc vòng đời hỗ trợ (EOL) từ đầu năm 2026 để chuyển hướng sang Grafana Alloy [3]. Trong phạm vi đồ án môn học, hệ thống vẫn triển khai đúng bộ công cụ Loki + Promtail theo yêu cầu bắt buộc của đề bài và ghi nhận rõ lộ trình nâng cấp sang Grafana Alloy trong phần định hướng phát triển.

4.5 Phương pháp kiểm chứng log

Để tạo dữ liệu nhật ký thực tế phục vụ truy vấn, kịch bản smoke test và thao tác trình duyệt đã thực hiện chuỗi hành động: truy cập các trang dự án/công việc hợp lệ (HTTP 200), gửi yêu cầu tới đường dẫn không tồn tại /not-found-page-xyz-999 (HTTP 404), gửi POST thiếu CSRF token (HTTP 403) và thực hiện đăng nhập với mật khẩu sai (HTTP 401 - sinh sự kiện login_failed). Sau khi Promtail đẩy log sang Loki, cả ba truy vấn LogQL được chạy kiểm chứng trực tiếp trên giao diện Grafana Explore.

4.6 LogQL xem log truy cập Nginx

```
{job="nginx"}
```

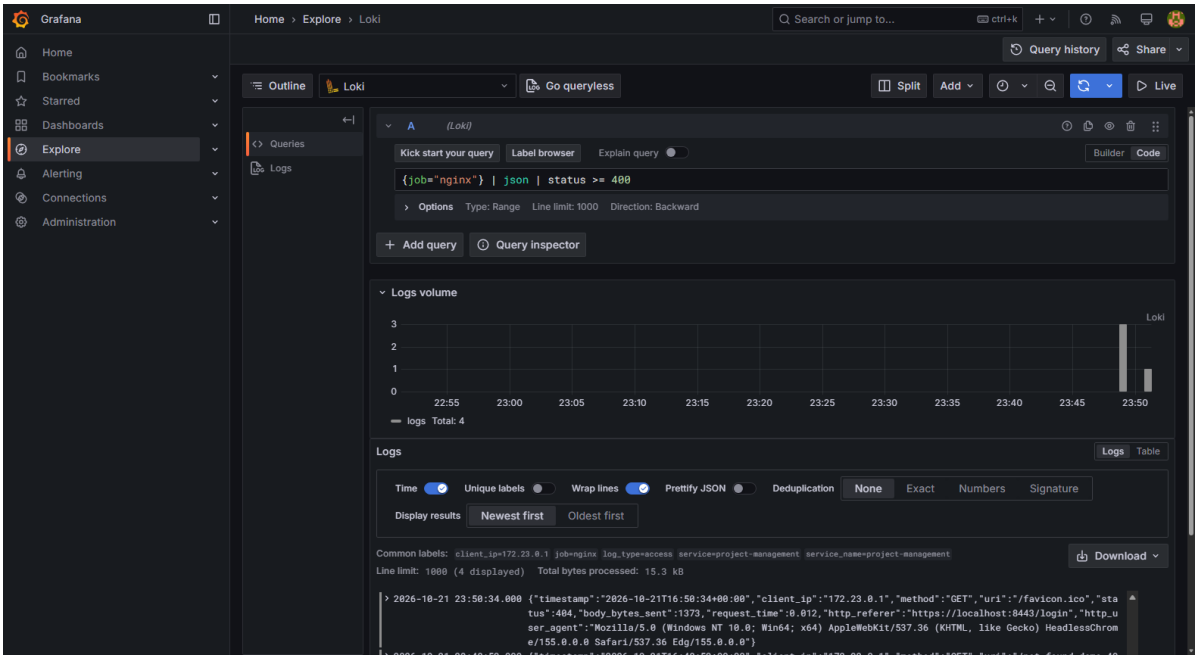


Hình 4.4 Truy vấn log truy cập Nginx trên Grafana Explore

Bộ chọn stream {job="nginx"} truy xuất toàn bộ nhật ký của Nginx reverse proxy đã được Promtail gắn nhãn. Kết quả trên Grafana Explore hiển thị biểu đồ tần suất log (Logs volume) cùng danh sách các dòng log JSON chứa đầy đủ thời điểm, địa chỉ IP, phương thức GET/POST, đường dẫn (/projects, /tasks, /health) và mã phản hồi HTTP thực tế.

4.7 LogQL lọc request lỗi HTTP

```
{job="nginx"} | json | status >= 400
```



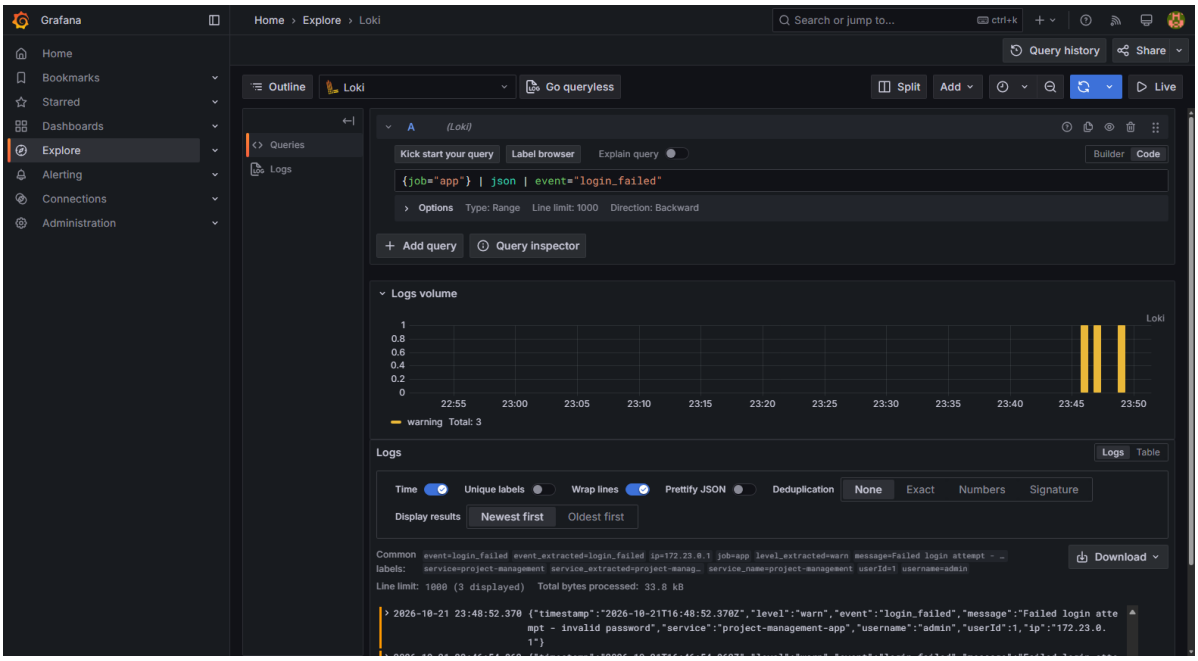
Hình 4.5 Lọc access log có mã HTTP từ 400 trở lên

Truy vấn thứ hai đưa luồng log Nginx qua bộ phân tích cú pháp | json để tách các trường dữ liệu, sau đó áp dụng điều kiện lọc số học status >= 400. Kết quả lọc chính xác các yêu cầu phát sinh mã lỗi phía client hoặc bị chặn bởi cơ chế bảo mật, bao gồm các yêu cầu

404 Not Found và 403 Forbidden khi kiểm thử CSRF, giúp quản trị viên nhanh chóng khoanh vùng các truy cập bất thường.

4.8 LogQL tìm sự kiện đăng nhập thất bại

```
{job="app"} | json | event="login_failed"
```



Hình 4.6 Tìm sự kiện login_failed của ứng dụng trong Loki

Truy vấn thứ ba chọn luồng nhật ký ứng dụng {job="app"}, giải mã JSON và lọc chính xác trường event="login_failed". Kết quả trả về các dòng cảnh báo mức warn ghi nhận thời điểm đăng nhập thất bại, tên tài khoản thử nghiệm (admin), mã phản hồi 401 và địa chỉ IP nguồn, nhưng hoàn toàn không chứa mật khẩu đã nhập. Truy vấn này giúp đội ngũ vận hành phát hiện sớm hành vi dò đoán mật khẩu mà vẫn đảm bảo an toàn dữ liệu nhạy cảm.

Query	**Sự kiện tạo thử**	**Kết quả ghi nhận trên Explore**
{job="nginx"}	Truy cập Dashboard, danh sách dự án, chi tiết dự án và healthcheck	Hiển thị đầy đủ các dòng access log JSON mã 200/302
{job="nginx"} | json | status >= 400	Truy cập đường dẫn không tồn tại (404) và gửi POST thiếu CSRF (403)	Lọc chính xác các dòng log có trường status bằng 403 và 404
{job="app"} | json | event="login_failed"	Đăng nhập với mật khẩu sai kèm CSRF token hợp lệ	Trả về đúng các bản ghi sự kiện login_failed mức warn (HTTP 401)

PHẦN 5 KIỂM THỬ VẬN HÀNH VÀ ĐỐI CHIẾU

5.1 Kết quả kiểm thử chức năng và tích hợp

Ca kiểm thử	Kết quả thực tế	Đánh giá
1. Chuyển hướng HTTP sang HTTPS	Gửi GET http://127.0.0.1:8080 nhận HTTP 301 sang https://127.0.0.1:8443/	Đạt
2. Healthcheck và kết nối DB	Endpoint /health trả HTTP 200, status UP, kết nối pm_db bằng tài khoản pm_app	Đạt
3. Kiểm tra Security Headers	Phản hồi HTTPS có đủ nosniff, X-Frame-Options: DENY và HSTS, không bị lặp header	Đạt
4. Kiểm soát truy cập ẩn danh	Truy cập /projects khi chưa đăng nhập bị chuyển hướng 302 về /login	Đạt
5. Sinh mã bảo vệ CSRF Token	Trang /login sinh đầy đủ trường ẩn _csrf gắn với phiên làm việc	Đạt
6. Đăng nhập sai và ghi log an ninh	Nhập sai mật khẩu trả HTTP 401; sinh sự kiện login_failed trong log và audit_logs	Đạt
7. Đăng nhập đúng tài khoản Admin	Xác thực thành công với tài khoản admin, tạo phiên hợp lệ và mở Dashboard	Đạt
8. Cờ bảo mật Session Cookie	Cookie pm_session_id có đủ cờ Secure, HttpOnly và SameSite=Lax	Đạt
9. Thao tác CRUD: Tạo dự án mới	POST /projects tạo thành công bản ghi dự án thử nghiệm trong PostgreSQL	Đạt
10. Thao tác CRUD: Xem chi tiết dự án	Trang chi tiết dự án hiển thị đúng mã, tên và thông tin vừa tạo	Đạt
11. Thao tác CRUD: Xóa dự án thử	POST xóa dự án thử nghiệm thành công, giữ nguyên các dự án mẫu ban đầu	Đạt
12. Chặn yêu cầu thiếu CSRF Token	Gửi POST không có trường _csrf bị từ chối ngay với mã HTTP 403 Forbidden	Đạt
13. Xử lý đường dẫn không tồn tại	Truy cập route lạ nhận trang lỗi HTTP 404 và ghi nhận vào access log	Đạt
14. Kiểm tra Prometheus Targets	API /api/v1/targets xác nhận cả 4 jobs (cadvisor, nginx, postgres, prometheus) đều UP	Đạt
15. Kiểm tra máy chủ Grafana	API /api/health của Grafana tại cổng 3002 trả HTTP 200, database: ok	Đạt

Bên cạnh bộ unit tests kiểm tra độc lập các hàm nghiệp vụ trong thư mục app/tests/, kịch bản scripts/smoke_test.py đã thực hiện kiểm thử tích hợp đầu-cuối (end-to-end) trực tiếp trên hệ thống container đang chạy qua giao thức HTTPS và ghi kết quả vào tệp docs/evidence/smoke_test_results.json với tỷ lệ đạt 15/15 (100%).

5.2 Backup phục hồi và bảo toàn dữ liệu

Kịch bản scripts/backup.py sử dụng công cụ pg_dump bên trong container pm-postgres để trích xuất toàn bộ cấu trúc và dữ liệu của cơ sở dữ liệu pm_db ra tệp SQL nằm trong thư mục backups/ (được loại trừ khỏi Git). Trong lần thực nghiệm, hệ thống đã tạo bản sao lưu backups/pm_db_backup_20261021_234722.sql (dung lượng 33.671 bytes). Để kiểm chứng khả năng phục hồi mà không làm ảnh hưởng hay ghi đè cơ sở dữ liệu đang vận hành, kịch bản scripts/restore_test.py tự động khởi tạo một cơ sở dữ liệu riêng biệt có tên pm_restore_test, nạp tệp SQL vừa sao lưu vào đó và đối chiếu số lượng bản ghi của từng bảng:

```
python scripts/backup.py
python scripts/restore_test.py
```

Bảng đối chiếu	Số bản ghi nguồn (pm_db)	Số bản ghi phục hồi (pm_restore_test)	Kết quả đối chiếu
users	3	3	Khớp 100%
members	5	5	Khớp 100%
projects	4	4	Khớp 100%
tasks	9	9	Khớp 100%
project_members	10	10	Khớp 100%
audit_logs	11	11	Khớp 100%

Kết quả kiểm chứng được lưu trong docs/evidence/restore_verification.json cho thấy toàn bộ 6 bảng dữ liệu nghiệp vụ trên cơ sở dữ liệu phục hồi thử nghiệm pm_restore_test có số lượng bản ghi khớp tuyệt đối với cơ sở dữ liệu gốc pm_db. Ngoài ra, khi thực hiện khởi động lại container pm-app và pm-postgres, toàn bộ dự án, công việc và thành viên vẫn được giữ nguyên nhờ cơ chế lưu trữ trên named volume pm_postgres_data.

5.3 Các lỗi gặp và cách xử lý

Hiện tượng	Nguyên nhân và cách xử lý thực tế
Cổng 3000/3001 trên host đã được sử dụng	Máy chủ đang chạy các container của một dự án khác; ánh xạ Grafana sang cổng 3002 (127.0.0.1:3002) và giữ nguyên toàn bộ container của dự án khác
Ổ đĩa C: hết dung lượng khi tải Docker image	Docker Desktop WSL2 giải nén nhiều layer lớn làm đầy ổ C:; di chuyển 9,8 GB tệp cài đặt tạm sang ổ E: để giải phóng bộ nhớ và khôi phục hoạt động Docker

Hiện tượng	Nguyên nhân và cách xử lý thực tế
pgAdmin 8.14 từ chối email mặc định đuôi .local	Phiên bản pgAdmin 8.14 kiểm tra nghiêm ngặt tên miền email; cập nhật biến PGADMIN_DEFAULT_EMAIL thành dtc245220019@ictu.edu.vn
Promtail không đọc được log Nginx mặc định	Image nginx:alpine trỏ symlink access.log/error.log vào /dev/stdout và /dev/stderr; dùng init-logs xóa symlink và tạo tệp vật lý thật trên volume pm_nginx_logs
Panel CPU/RAM container trên Grafana báo No data	cAdvisor v0.51.0 không đọc được nhãn name trên Docker Engine 29 dùng containerd snapshotter; nâng cấp lên ghcr.io/google/cadvisor:0.55.1 và mount /var/lib/containerd:ro

5.4 Đối chiếu yêu cầu và kịch bản demo

Yêu cầu đề bài	Minh chứng trong báo cáo
Quản lý mã nguồn trên GitHub	Hình 2.1, 2.2; ba commit mốc kỹ thuật, README hướng dẫn tái lập và cấu hình .gitignore
Ứng dụng Quản lý Dự án, PostgreSQL và pgAdmin	Hình 2.3 đến 2.10; đầy đủ chức năng dự án, công việc, thành viên, audit logs và 7 bảng trong pgAdmin 4
Docker Compose và Nginx reverse proxy	Mục 2.3, 3.1 và 3.2; điều phối đa container, proxy_pass, HTTPS tự ký RSA 3072 bit và security headers
Giám sát với Prometheus và Grafana	Hình 4.1 đến 4.3; 4 targets UP, metrics dịch vụ web/DB và đồ thị tài nguyên container thực tế
Thu thập log với Loki, Promtail và 3 LogQL	Hình 4.4 đến 4.6; log JSON của Nginx/app và 3 truy vấn LogQL trả về dữ liệu thật
Hardening hệ thống (tối thiểu 4 biện pháp)	Mục 3.3 và 3.4; đã áp dụng và kiểm chứng 8 biện pháp (non-root, read-only, cap_drop, internal net, least privilege...)
Kiểm thử, sao lưu phục hồi và báo cáo hoàn chỉnh	Mục 5.1 và 5.2; 15/15 smoke test PASS, backup/restore vào DB riêng khớp 100%, báo cáo đúng chuẩn mẫu

Kịch bản trình diễn (demo) trong buổi bảo vệ được thực hiện tuần tự qua 6 bước: (1) Mở trang GitHub repository giới thiệu cấu trúc mã nguồn, tài liệu README và lịch sử các commit mốc; (2) Truy cập http://localhost:8080 để chứng minh Nginx tự động chuyển hướng sang https://localhost:8443, đăng nhập tài khoản quản trị, thực hiện thêm/cập nhật dự án cùng công việc và kiểm tra vết trong trang Audit Logs; (3) Mở pgAdmin 4 tại http://localhost:8081 đối chiếu trực tiếp dữ liệu trong 7 bảng của cơ sở dữ liệu pm_db; (4) Mở Prometheus tại http://localhost:9090/targets xác nhận 4 targets UP và mở Grafana Dashboard tại http://localhost:3002 trình bày các đồ thị CPU, RAM, mạng, lưu lượng Nginx và kết nối PostgreSQL; (5) Chuyển sang Grafana Explore chạy trực tiếp 3 câu truy vấn LogQL trên nguồn dữ liệu Loki; (6) Chạy nhanh hai kịch bản kiểm chứng

verify_hardening.py và restore_test.py trên cửa sổ dòng lệnh mà không mở tệp .env hay làm lộ mật khẩu.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Đề tài số 14 đã triển khai hoàn chỉnh Hệ thống Quản lý Dự án (Project Management) đáp ứng đầy đủ các chức năng quản trị dự án, phân công công việc, quản lý thành viên và nhật ký kiểm toán trên nền tảng Docker Compose, kết nối cơ sở dữ liệu PostgreSQL 16 và quản trị qua pgAdmin 4. Nginx đảm nhiệm vai trò reverse proxy bảo mật với chứng chỉ HTTPS tự ký RSA 3072 bit và các HTTP security headers chuẩn mực. Toàn bộ mã nguồn, tệp cấu hình và tài liệu hướng dẫn đã được quản lý có hệ thống trên GitHub với các commit mốc rõ ràng.

Hệ thống quan sát thu thập thành công các chỉ số tài nguyên phần cứng container (qua cAdvisor 0.55.1), lưu lượng web server (qua Nginx exporter) và hoạt động cơ sở dữ liệu (qua PostgreSQL exporter). Bốn mục tiêu trên Prometheus đều đạt trạng thái UP và hiển thị dữ liệu thời gian thực trên Grafana dashboard được provision tự động. Ngăn xếp Loki và Promtail thu gom hiệu quả nhật ký JSON từ ứng dụng và Nginx, đáp ứng chính xác cả ba truy vấn LogQL phục vụ theo dõi truy cập, lọc lỗi HTTP và phát hiện sự kiện đăng nhập thất bại. Các lớp hardening bảo mật cùng quy trình sao lưu, phục hồi thử vào cơ sở dữ liệu độc lập pm_restore_test đã được kiểm chứng toàn diện.

Bên cạnh các kết quả đạt được, hệ thống trong phạm vi môn học vẫn còn một số giới hạn thực tế: ứng dụng hiện chạy một instance duy nhất với bộ đếm rate limit lưu trong bộ nhớ RAM; chứng chỉ TLS là chứng chỉ tự ký chỉ phục vụ môi trường demo cục bộ; các quy tắc cảnh báo trên Prometheus chưa kết nối tới kênh gửi thông báo tự động ra bên ngoài (như Alertmanager qua Email/Telegram); tệp nhật ký trên volume chưa thiết lập cơ chế xoay vòng (log rotation) tự động và tác tử Promtail đã bước vào giai đoạn EOL theo lộ trình của nhà phát triển.

Hướng phát triển tiếp theo của đề tài là tích hợp công cụ xoay vòng nhật ký (logrotate), nâng cấp tác tử thu gom log từ Promtail sang Grafana Alloy, cấu hình Alertmanager để chủ động gửi cảnh báo khi dịch vụ gián đoạn, sử dụng chứng chỉ số từ CA tin cậy khi triển khai trên tên miền thực tế và xây dựng quy trình CI/CD tự động quét lỗ hổng bảo mật cho các Docker image trước khi phát hành.

TÀI LIỆU THAM KHẢO

[1] Docker. Control startup and shutdown order in Compose.

https://docs.docker.com/compose/how-tos/startup-order/

[2] Docker. Compose file reference.

https://docs.docker.com/reference/compose-file/

[3] Grafana Labs. Promtail agent và thông báo EOL.

https://grafana.com/docs/loki/latest/send-data/promtail/

[4] Grafana Labs. LogQL log queries.

https://grafana.com/docs/loki/latest/query/log_queries/

[5] Prometheus. Querying basics.

https://prometheus.io/docs/prometheus/latest/querying/basics/

[6] Nginx. HTTP proxy module.

https://nginx.org/en/docs/http/ngx_http_proxy_module.html

[7] PostgreSQL Global Development Group. PostgreSQL 16 Documentation.

https://www.postgresql.org/docs/16/

[8] pgAdmin Development Team. pgAdmin 4 Container Deployment.

https://www.pgadmin.org/docs/pgadmin4/latest/container_deployment.html

[9] Google. cAdvisor container resource monitoring.

https://github.com/google/cadvisor

[10] Đinh Gia Bảo. Mã nguồn và cấu hình Hệ thống Quản lý Dự án Đề 14.

https://github.com/dtc245220019-create/ProjectManagement_TKQTHT

Tài liệu yêu cầu học phần: Đề 14 Hệ thống Quản lý Dự án (Project Management); yêu cầu triển khai bằng Docker Compose, quản lý mã nguồn trên GitHub, cơ sở dữ liệu PostgreSQL và pgAdmin, Nginx reverse proxy, giám sát Prometheus/Grafana, quản lý nhật ký Loki/Promtail với LogQL và hardening bảo mật. Các kết quả thực nghiệm trong báo cáo được kiểm chứng ngày 21–22/10/2026.